

Foundations of Cybersecurity / Applied Cryptography

07 February 2023

Exercise n. 1 [10]

With reference to one-time pad (OTP), answer the following questions.

1. Illustrate the key generation, encryption, and decryption algorithm.
2. Given $k_0 = 0^n$, then $c = p \oplus k_0 = p$.¹ Argue whether removing k_0 from the key set is a good idea or not.
3. Let $c = \text{"HELLO"}$. Specify which of the following strings could constitute possible corresponding plaintexts and which cannot be.
 - a. "HELLO"
 - b. "LIGHT"
 - c. "BLUE"
 - d. "YELLOW"

Exercise n. 2 [12]

Let us consider the DES cipher in the CBC mode of operation. Assuming we encrypt a very large amount of data with the same key, it is possible that two different ciphertext blocks, say c_i and c_j , with $i \neq j$, are the same.

1. Does the fact that $c_i = c_j$ leak any information regarding the plaintext blocks m_i and m_j ?
2. Does replacing DES with 3DES avoid the information leakage?
3. How can we mitigate this attack?

Exercise n. 3 [8]

Find, explain, and patch the vulnerabilities. If any, of the following function. The function expects a user to open a file in the `/home/Alice/` path, from where it is executed.

```
void do_stuff(const char* file) {
    if (strlen(file) > 0 && file[0] == '/')
        { /* Absolute path! Handle error */ }
    FILE *f = fopen(file, "r");
    if (!f) { /* File doesn't exists, handle error */ }
    else {
        /* read file*/
    }
    fclose(f);
}
```

¹ By 0^n we denote a string of n bits where each bit is equal to 0.

Solution

Exercise n.1

1. See theory.
2. It is a bad idea because removing k_0 the resulting cipher is not perfect anymore. The number of keys gets smaller than the number of messages so violating the necessary condition of the Shannon's Theorem.
3. Strings a) and b) are possible plaintexts with the same probability. Strings c) and d) cannot be possible plaintext because they are, respectively, shorter, and longer than any possible plaintext.

Exercise n.2

1. If $c_i = c_j, i \neq j$, then $E_k(c_{i-1} \oplus p_i) = E_k(c_{j-1} \oplus p_j)$. It follows that we obtain the differential information $p_i \oplus p_j = c_{i-1} \oplus c_{j-1}$. Therefore, for example, it is easy to determine whether two plaintexts are equal. If $p_i = p_j$ the $p_i \oplus p_j = 0$, so $c_{i-1} \oplus c_{j-1} = 0$, and thus $c_{i-1} = c_{j-1}$.
2. Using 3DES does not bring any advantage because the 3DES block size is the same as DES.
3. A mitigation may be introduced using a block cipher with a larger block size (e.g., AES) so that the probability that two ciphertext are equal is smaller.

Exercise n.3

The function displays an instance of the classic *canonicalization* problem that can be addressed by means of the `realpath()` function. The path returned by the `realpath` function is absolute. According to assumptions, the `do_stuff` function can then check whether the returned absolute task contains `/home/Alice`.

Furthermore, the function also displays *TOCTOU* problems.

A possible patch is:

```
void do_stuff(const char* file) {
    char* canon_file = malloc(/*appropriate size*/);
    canon_file = realpath(file, NULL);
    if(!canon_file){/*handle error*/}
    const char[] ok_dir = "/home/alice";
    if(strncmp(canon_file, ok_dir, strlen(ok_dir))!=0)
        /*handle error, path unauthorized*/}
    FILE *f = fopen(file, "r");
    if (!f) { /* File doesn't exists, handle error */ }
    else {
        /*read file*/
    }
    fclose(f);
    free(canon_file);
}
```

Remember that the `realpath()` function derives, from the path name pointed to by `file_name`, an absolute path name that names the same file, whose resolution does not involve `"."`, `".."`, or symbolic links. Notice that a canonical path is always an absolute path. Converting from a path to a canonical path makes it absolute (usually tack on the current working directory so e.g. `./file.txt` becomes `c:/temp/file.txt`).